

# Resumo — Comandos essenciais

---

**git init**

Inicializa o repositório

**git status**

Estado atual dos arquivos

**git add .**

Prepara todos para commit

**git commit -m "msg"**

Cria o commit

**git log --oneline**

Histórico compacto

**.gitignore**

Ignora arquivos

**git rm --cached**

Remove do rastreamento

**git push -u origin main**

Envia commits para o GitHub

# Git Colaborativo

Branches

Fetch

Pull

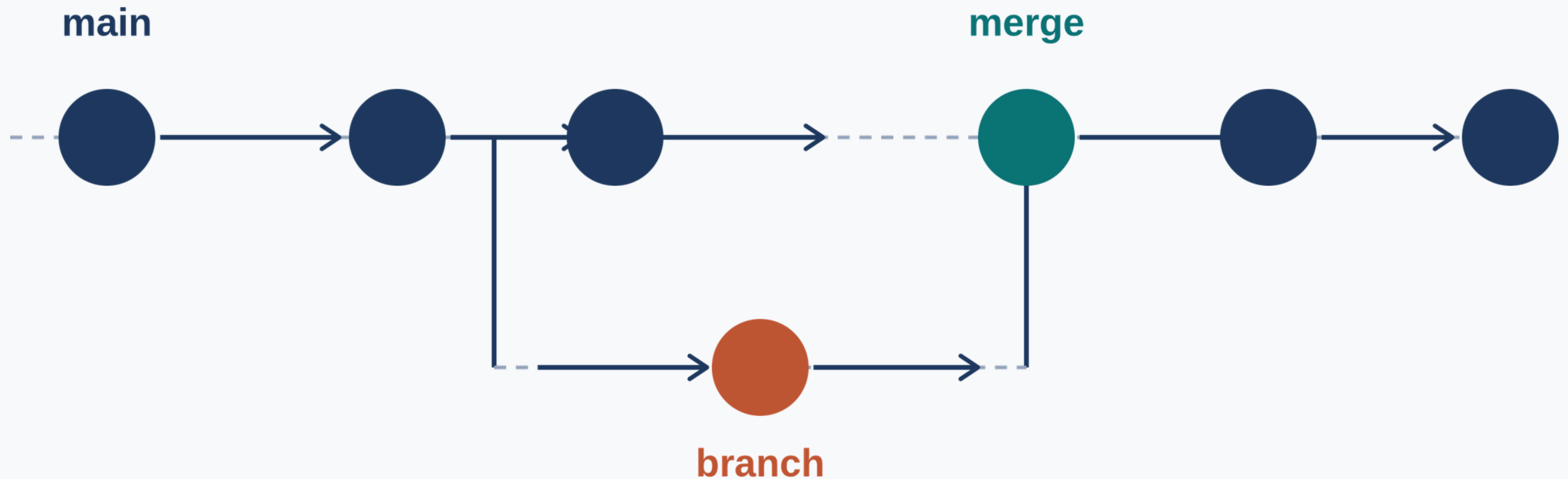
Merge

Pull Request

# O que é uma Branch?

---

Branch é uma linha paralela de desenvolvimento.



# Criar e trocar de Branch

---

**git branch nomeBranch**

Cria uma branch

**git checkout nomeBranch**

Entra na branch

**git checkout -b nomeBranch**

Cria e entra na branch

**git switch nomeBranch**

Troca para uma branch existente

**git branch**

Lista todas as branches (atual marcada com \*)

# Trabalhando em uma Branch

---

## main

Versão estável e em produção

Não deve ser alterada diretamente

Só recebe código via merge

## branchNome

Nova funcionalidade em desenvolvimento

Alterações não afetam a main

Vai para a main após aprovação via merge

*Branch permite experimentar sem quebrar o código principal.*



# git fetch

---

Busca atualizações do GitHub, mas não aplica nada no seu código.

✓ Baixa novos commits e branches remotas

✓ Seu código permanece exatamente igual

✓ Serve para analisar antes de decidir

**git fetch = buscar e analisar. Sem risco.**

# git pull

---

Busca as atualizações do GitHub E já aplica no código local.

## git fetch

Busca, mas não aplica.  
Você decide quando integrar.

## git pull

Busca E aplica automaticamente.  
Equivalente a fetch + merge.

**Sempre faça git pull antes de começar a trabalhar no projeto.**

# Unindo branches com git merge

---

*Terminei minha funcionalidade. Como juntar com a main?*

O git merge une as alterações de uma branch na branch atual.

```
# 1. Volte para a main
git checkout main      ou      git switch main

# 2. Faça o merge
git merge feature-login
```

*Normalmente usado quando a funcionalidade está pronta e aprovada.*



# Pull Request (PR)

---

PR = pedido formal para incorporar alterações na main com revisão de código.

1

## Envie a branch

git push origin branchNome

2

## Abra o PR no GitHub

Clique em "Compare & Pull Request"

3

## Descreva as mudanças

Solicite revisão ao time

4

## Revisor aprova

Revisão de código

5

## Merge pull request

Código integrado à main

# Deletar Branch

---

Depois do merge, a branch não é mais necessária.

**Deleta localmente (só se já houve merge):**

```
git branch -d nomeBranch
```

**Força a exclusão mesmo sem merge:**

```
git branch -D nomeBranch
```

**Remove do GitHub (remoto):**

```
git push origin --delete nomeBranch
```

*Boa prática: manter o repositório limpo deletando branches após o merge.*

# Prática

1

## Criar a branch

`git checkout -b nomeBranch`

2

## Fazer alterações

Edite arquivos do projeto

3

## Commit e push

`git add . → git commit → git push origin nomeBranch`

4

## Abrir PR

No GitHub: "Compare & Pull Request"

5

## Revisão

Solicite revisão a um colega

6

## Merge e deletar branch

Aprovar, fazer merge e deletar a branch

# Fluxo completo do Git

## Projeto Individual

```
git init
```

```
git add .
```

```
git commit -m
```

```
git push
```

## Trabalho em Equipe

```
git checkout -b
```

```
commits
```

```
git push origin
```

```
PR + Merge
```

*Esse é o fluxo básico de um projeto colaborativo com Git e GitHub.*

# Resumo Final

---

## Projeto Individual

```
git init
```

```
git status
```

```
git add .
```

```
git commit -m "mensagem"
```

```
git log
```

```
git push
```

## Trabalho em Equipe

```
git branch
```

```
git checkout -b
```

```
git fetch
```

```
git pull
```

```
git merge
```

```
git push origin --delete
```